

Topic 2 Working with files

igor.razgon@durham.ac.uk

Content

1. Type of files
2. Built-in functions for file operations
3. TXT File Reading and Writing
4. CSV and Excel files: brief overview
5. Working with ZIP files

Type of files

text file (.TXT):

- reading and writing line by line
- files that humans can read "by eye"
- a standard text document that contains unformatted text.

```
MPL 40
  pattern1 = 786
  pattern2 = 1
  pattern3 = 979
  pattern4 = 0
```

Example of txt file

```
<FF>0<FF><E0>^@^
PJFIF^@^A^B^A^@<9
6>^@<96>^@^@<FF
><E0>~JFXX^@^P<F
F>00<FF>U^@^C^@^
```

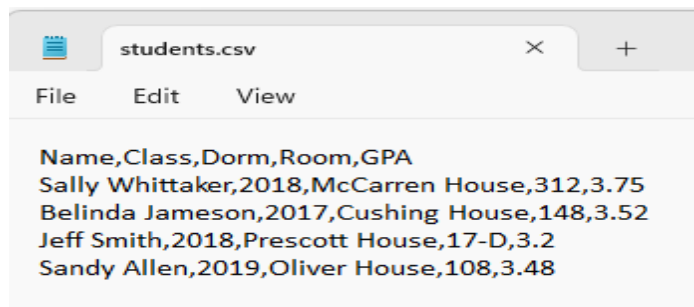
Not a text file
Binary file example
(File of image)

CSV /Excel files

CSV stands for "**comma-separated values**". Its data fields are most often separated, or delimited, by a comma.

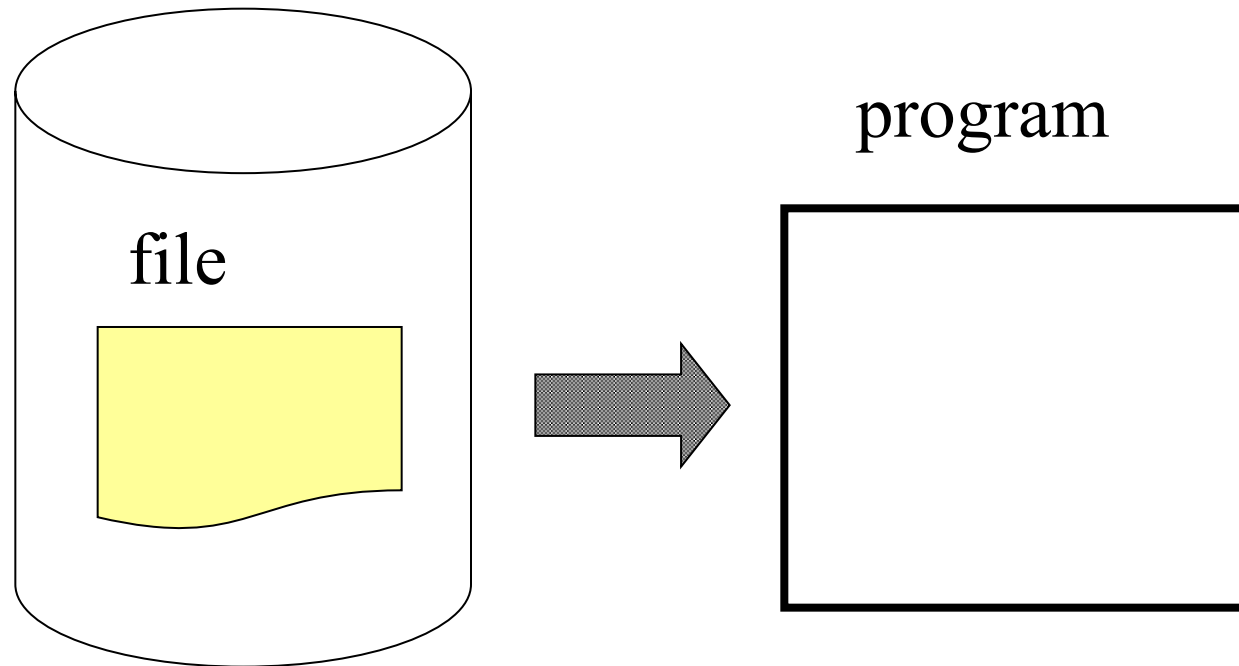
CSV is a simple file format used to store [tabular](#) data, such as a [spreadsheet](#) or [database](#). Files in the CSV format can be [imported](#) to and [exported](#) from programs that store data in tables, such as [Microsoft Excel](#) .

Name	Class	Dorm	Room	GPA
Sally Whittaker	2018	McCarren House	312	3.75
Belinda Jameson	2017	Cushing House	148	3.52
Jeff Smith	2018	Prescott House	17-D	3.20
Sandy Allen	2019	Oliver House	108	3.48



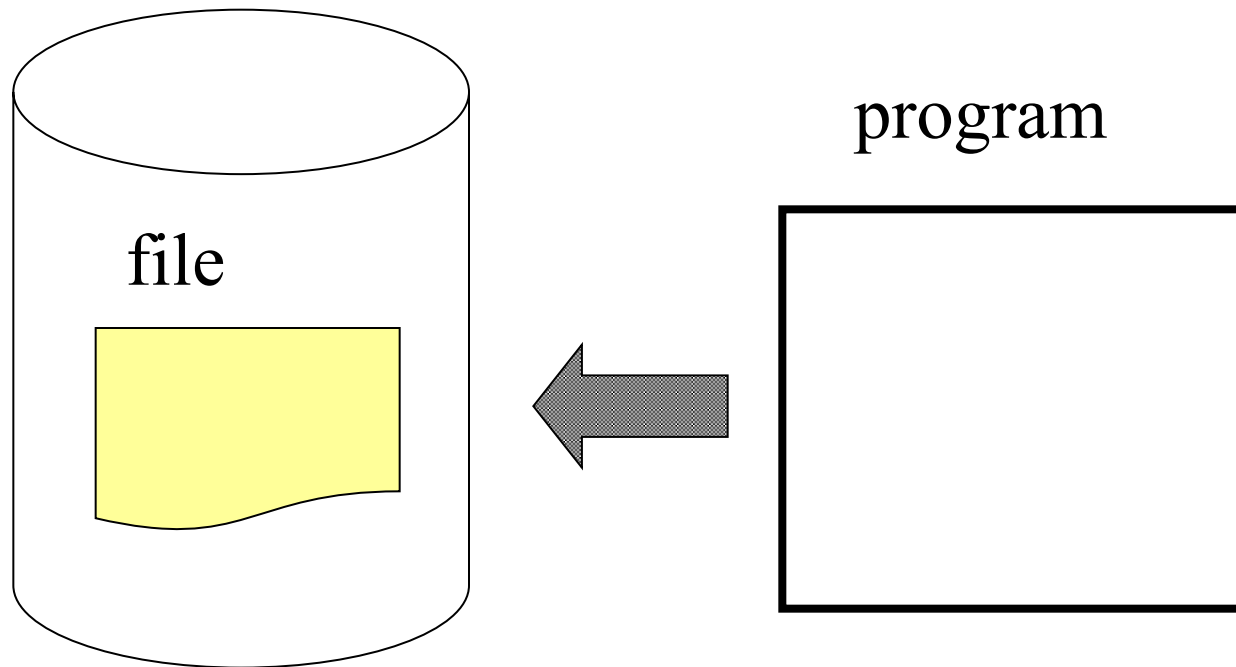
```
students.csv
File Edit View
Name,Class,Dorm,Room,GPA
Sally Whittaker,2018,McCarren House,312,3.75
Belinda Jameson,2017,Cushing House,148,3.52
Jeff Smith,2018,Prescott House,17-D,3.2
Sandy Allen,2019,Oliver House,108,3.48
```

read from a file



- The content of the file does not change

write to a file



- Content of the file changes
- Files may grow and shrink

Built-in functions for file operations

➤ File open and close

- `f = open (" file name. extension ", mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None)`

➤ Specify the operation

"r"	read. Show an error if the file does not exist
"w"	write (overwrite!). Create if file does not exist
"a"	append (at the end) write. Create if file does not exist
"r+"	both read and write. Show an error if file does not exist
"w+"	both read and write. Create if file does not exist

- You have to close the file after reading and writing the file.
Use `f.close ()` to closing at the end (if not using memory)

Built-in functions for file operations

➤ read from a file (“r”)

`f.read()` Read all the contents of the file as one string
`f.readline()` Single line reading (including a newline character (\n))
Return empty string when finished reading
`f.readlines()` read all the lines of a file in a list

➤ write to a file (“w”)

`f.write(string)`
writes the contents of string to the file,
returning the number of characters written.

`f.writelines(sequence)`

writes a sequence of strings to the file.

The sequence can be any iterable object producing strings,
typically a list of strings. There is no return value.

```
f=open("text1.txt","r")
s = f.readline()
print(s)
s = f.readline()
print(s)
f.close()
```

➤ append to a file (“a”)

Using “a” instead of “w” to specify the operation

TXT File Reading and Writing

A following roster file (text file format)

- Each data is separated by single-byte space character (s)

```
Taro 1200/01/01 Japan  
Jiro 1300/12/31 USA  
Hanako 1800/05/31 UK
```

3-line text file

Save as "test1.txt"

Simple read from a file

```
filename="test1.txt"    #file name
f=open(filename)        #Open a file with a file name
data = f.read()         # Assign file contents to data
print(data)
f.close()               #close file
```

Taro 1200/01/01 Japan

Jiro 1300/12/31 USA

Hanako 1800/05/31 UK

✕open (filename) is an
abbreviation for open
(filename, r)

Read from a file using “with”

```
filename="test1.txt"  
with open(filename) as f:  
    data = f.read()  
    print(data)
```

} Open the file and execute the following block, file will be closed automatically

```
Taro 1200/01/01 Japan  
Jiro 1300/12/31 USA  
Hanako 1800/05/31 UK
```

✕ It will automatically close the file if an error occurs in the block.

If the file does not exist at the time of reading

#specific a file that doesn't exist

```
1 filename="test3.txt"
2 with open(filename,"r") as f:
3     data=f.read()
4
```

Running: 11 files.py

Traceback (most recent call last):

File "c:\users\wang jingyun\desktop\プログラミング演習python\english
2019-2020\2020ppt\class py files\11 files.py", line 2, in <module>

with open(filename,"r") as f:

FileNotFoundError: [Errno 2] No such file or directory: 'test3.txt'

>>> |

Write to a file

```
filename="test.txt"
with open(filename, "w") as f: #Specify a file and write
    f.write("Keiko 2100/06/20 France")
    ✕If specified file does not exist, generate one
with open(filename, "r") as f:
    print(f.read()) # Check if it is generated properly
```

Keiko 2100/06/20 France

A little more complicated text processing

Consider a text file marksinformal.txt with the following content

Alice did very well, her mark was 96

Bob could have done better, his mark was 80

Carol did not do really well, her mark was 70

Daniel, on the other hand, did great. His mark was 97

We see that the first word of each line is a student name and the last word is their mark.

We want to print just the name and the mark ignoring the rest of the text.

Processing the text with student marks

with open('marksinformal.txt','r') as mf:

for line in mf:

words=line.split()

print(words[0]+' '+words[len(words)-1])

The output of the above program is

Alice 96

Bob 80

Carol 70

Daniel, 97

A basic NLP task

- The program we have just considered was a Natural Language Processing task.
- Note that even with such heavy assumptions about the input, the output was not perfect
- For example, there was a comma after name Daniel

Accessing .CSV and Excel files

- .csv and Excel files can be neatly accessed through Pandas.
- We will consider their processing on the next Friday.

Extracting Data from ZIP Files

<https://docs.python.org/3/library/zipfile.html>

<https://www.geeksforgeeks.org/working-zip-files-python/>

ZIP and zipfile

ZIP is an archive file format that supports **lossless data compression**. A ZIP file may contain one or more files or directories that may have been compressed. The ZIP file format permits a number of compression algorithms, though DEFLATE is the most common. This format was originally created in 1989 and was first implemented in PKWARE, Inc.'s PKZIP utility, as a replacement for the previous ARC compression format by Thom Henderson. [https://en.wikipedia.org/wiki/ZIP_\(file_format\)](https://en.wikipedia.org/wiki/ZIP_(file_format))

The zipfile module provides tools to create, read, write, append, and list a ZIP file. Any advanced use of this module will require an understanding of the format, as defined in [PKZIP Application Note](#).

Example of Zip file processing in Python

- We will write a program for processing a Zip archive containing **text files** only.
- The program merges the contents of all the files in the archive into a single list whose elements are the lines of all the files.
- For example, suppose that the archive contains two files
file1.txt and file2.txt
- The content of file1.txt is
line1
line2
- The content of file2.txt is
line3
line4
- Then the program will create the list [*'line1'*, *'line2'*, *'line3'*, *'line4'*]

Preparing the archive

We first do the preparation: import the *ZipFile* class and prepare the files for the archive.

```
from zipfile import ZipFile  
with ZipFile('example.zip', 'w') as myzip:  
    myzip.write('file1.txt')  
    myzip.write('file2.txt')  
myzip.close()
```

The main part of the program

with ZipFile('example.zip', 'r') as myzip:

#List of file names in the archive

files=myzip.namelist()

print(files)

#The text lines of all files to be gathered

##into a single list

textlines=[]

for file in files:

extrdata=myzip.read(file)

print(extrdata)

#The data in the archived file is stored

#in the byte format, it needs to be decoded back to utf-8

origdata=extrdata.decode('utf-8')

origlines=origdata.splitlines()

textlines=textlines+origlines

myzip.close

The output of the program

- The program prints the following information
 - ['file1.txt', 'file2.txt']* the list of files in the archive
 - b'line1\r\nline2\r\n'* the content of file1.txt in bytecode
 - b'line3\r\nline4\r\n'* the content of file2.txt in bytecode
- The content of the list *textlines* by the end of the program:
['line1', 'line2', 'line3', 'line4']